🧩 1. What is a Class?

A **Class** is like a **blueprint** or **template** used to create objects.

Think of a **class** as a **design of a house** —
you can build **many houses (objects)** from the same design.

🧱 2. What is an Object?

An **object** is a **real-world entity** created (or instantiated) from a class.
It has:

- **Attributes (variables)** → describe the object
- **Methods (functions)** → define behavior

📐 3. Defining a Class

Let's define a simple class called Car.

```
# Example 1: Class and Object


class Car:
    # Constructor: initializes the object
    def __init__(self, brand, model, year):
        self.brand = brand   # attribute
        self.model = model   # attribute
        self.year = year     # attribute

    # Method: defines behavior
    def start_engine(self):
        print(f"{self.brand} {self.model}'s engine started!")
```

```
    def display_info(self):
        print(f"Car: {self.brand}, Model: {self.model}, Year: {self.year}")
```

🚙 4. Creating Objects

Now let's create multiple cars using this class.

```
# Creating objects
car1 = Car("Toyota", "Corolla", 2020)
car2 = Car("Tesla", "Model S", 2023)

# Accessing attributes
print(car1.brand)  # Toyota
print(car2.model)  # Model S

# Calling methods
car1.start_engine()
car2.display_info()
```

🧠 **Output:**

```
Toyota Corolla's engine started!
Car: Tesla, Model: Model S, Year: 2023
```

⚙️ 5. Understanding self

self refers to the **current object** —
It's how Python differentiates between instance attributes and local variables.

Example:

```
def __init__(self, brand):
    self.brand = brand
```

Here self.brand → attribute belongs to the object
brand → argument passed while creating object

💡 6. Adding and Accessing Attributes Dynamically

You can also add attributes **after** object creation.

```
car1.color = "Red"

print(car1.color)  # Red
```

📝 But this is **not recommended** — always define attributes inside the constructor for better structure.

🔄 7. Real-World Analogy

Real World	Python Equivalent
Blueprint	Class
Actual House	Object
Features (doors, windows)	Attributes
Actions (open door)	Methods

🧪 8. Practice Example

Try this yourself 👇
(Write it in your editor and run — don't worry about errors, that's how you learn)

```
# Exercise: Class and Object

# Define a class called 'Student'

# - attributes: name, grade, roll_number

# - methods: display_info() that prints the student details


# Create 2 objects of Student and call the method
```

🗒️ *Hint: Use a constructor __init__ to initialize attributes.*

🔍 9. Summary

Concept	Description	Example
Class	Blueprint for creating objects	class Car:
Object	Instance of a class	car1 = Car()
Constructor (__init__)	Initializes object attributes	def __init__(self, ...):
self	Refers to current object	self.brand = brand
Method	Function inside class	def start_engine(self):